WNDRVR
SUPPORT NETWORK

VXWORKS 7 IEEE 1394 SERIAL BUS PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 IEEE 1394 Serial Bus Programmer's Guide, SR0650

10 June 2020

1. OVERVIEW

1. About VxWorks and the IEEE 1394 Serial Bus

The VxWorks® 7 IEEE 1394 Serial Bus Programmer's Guide describes the VxWorks support for the IEEE 1394 Serial Bus.

VxWorks conforms to IEEE 1394-1995, 1394a-2000, 1394b-2002, and 1394-OHCI v1.1 standards for the IEEE 1394 serial bus. VxWorks provides a set of ioctl( ) commands that allow applications to communicate with and control IEEE 1394 devices, as well as shell routines and other facilities for monitoring and debugging activities.

The IEEE 1394 serial bus is often referred to by the Apple trademark FireWire. In this document, however, it will be referred to by the name of the IEEE standard or an abbreviation thereof (for example, “1394 bus”).

2. Wind River’s IEEE 1394 Core Stack

The Wind River IEEE 1394 core stack supports the interface to a VxWorks I/O system driver that implements the core functionality.

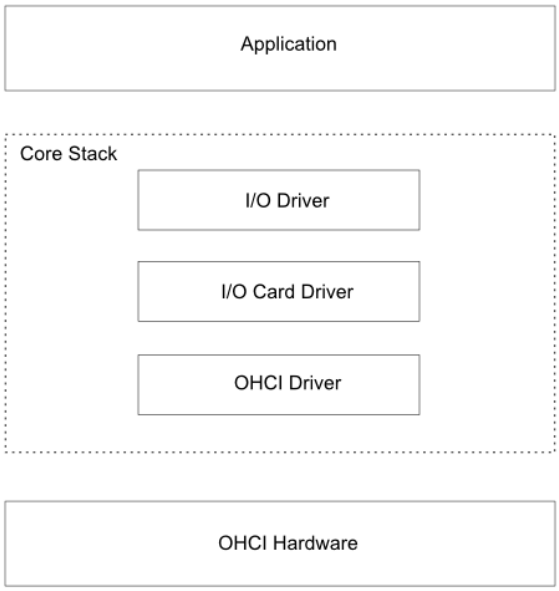
It does not provide an interface to typical IEEE 1394 devices such as video cameras, disks, and so on. To implement higher level functionality, make use of the **ioctl( )** commands provided by the core stack (for information in this regard, see Coding Applications on page 7 and ioctl( ) Command Reference on page 9). For example, RFC 2734 and SBP-2 implementations can be based on the core stack driver.

Architecture Overview on page 1 The core stack consists of three layers: the I/O driver layer, the I/O card driver layer, and the OHCI driver layer.
Initialization Process on page 2 VxBus and I/O drivers must be initialized to use IEEE 1394.

2.1. Architecture Overview

The core stack consists of three layers: the I/O driver layer, the I/O card driver layer, and the OHCI driver layer.

Figure 1. Wind River IEEE 1394 Architecture



The I/O driver layer supports the interface to applications. It implements all ioctl( ) commands in the application interface and calls I/O card driver routines.

The I/O card driver layer connects the I/O driver interface to the OHCI driver. It calls the driver ioctl( ) commands, which are registered by way of the OHCI driver with VxBus, using the name **driverControl**.

The OHCI driver layer is the device driver for OHCI hardware. It initializes the OHCI hardware controller, registers the driver with VxBus system, and provides the interface to the upper management layer. It includes a bus management task, an asynchronous transfer management task, and isochronous transfer management task. The bus management task is responsible for handling bus resets, managing a list of nodes, and managing each node's configuration ROM. Asynchronous and isochronous management tasks primarily deal with their data transfer contexts.

Parent topic: Wind River's IEEE 1394 Core Stack on page 1

2.2. Initialization Process

VxBus and I/O drivers must be initialized to use IEEE 1394.

VxBus Driver Initialization

VxBus Driver initialization involves the following steps:

1. The probe function is called to identify the supported OCHI chips.
2. Chip registers are mapped.
3. Memory is allocated for internal structures and DMA.
4. OHCI hardware is initialized.
5. The bus manager is initialized.
6. The context manager is initialized.
7. The controller GUID is read.
8. The configuration ROM is initialized.
9. The IEEE1394 bus is reset.

I/O Driver Initialization

I/O Driver initialization involves the following steps:

1. Data structures are allocated and initialized.
2. A bus reset callback is installed.
3. When a bus reset occurs, the I/O table entries are updated or created based on the new topology. I/O table entries for removed nodes are deleted.

Parent topic: Wind River's IEEE 1394 Core Stack on page 1

3. Data Flow

The Wind River IEEE 1394 core stack data flow is implemented using the fwIoCtl( ), fwCardIoctl( ), fwOhciDriverCtrl( ), and callback( ) functions.

WNDRVR

Figure 2. Core Stack Data Flow

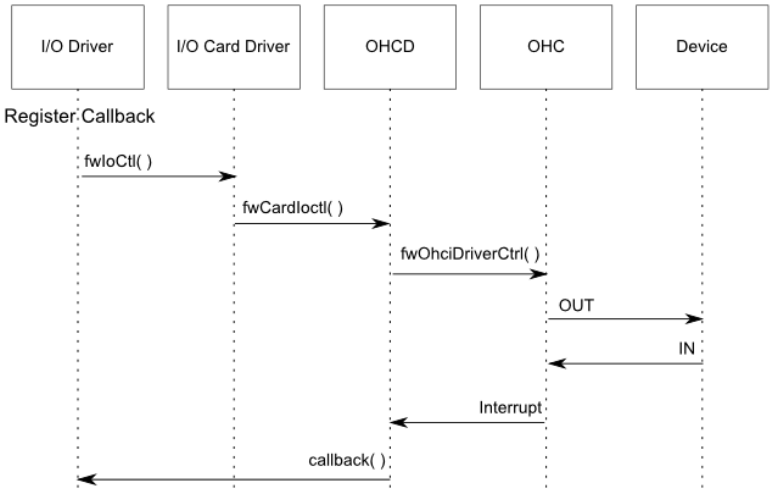
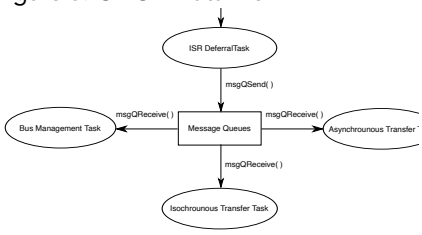


Figure 3. OHCD Data Flow



4. Scope of Support

The core stack functionality includes support for key aspects of the

- Asynchronous support: read/write/lock, and asynchronous stream.
- Isochronous support: a basic Tx/Rx transfer with packet-per-buffer mode, and set header/no-header mode.
- Bus management.
- Isochronous resource management.
- Bus reset.
- Get ROM configuration.
- Get GUID.

2. CONFIGURING VXWORKS

1. Configuring VxWorks with 1394 Serial Bus Support

VxWorks does not support the 1394 serial bus by default. You must add the layer to the VxWorks source build (VSB) project, add the required components to the VxWorks Image Project (VIP), and rebuild VxWorks.

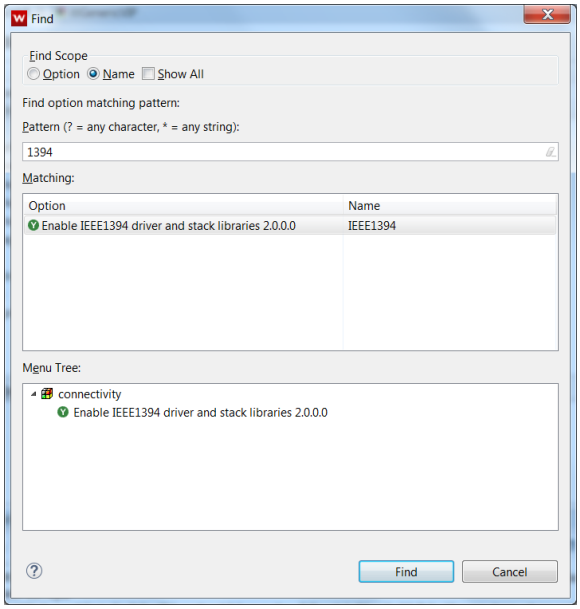
About This Task

These steps add the 1394 serial bus to VxWorks. They assume the following:

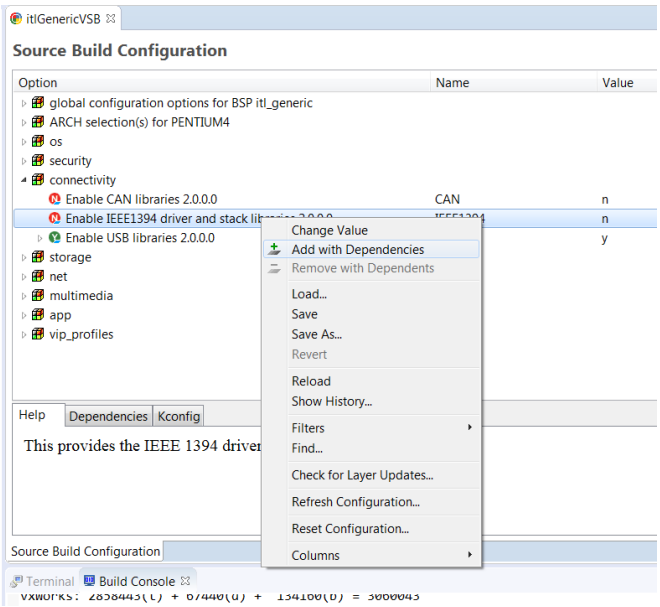
1. you are familiar with creating and building both VSB and VIPs from Workbench. For more information, refer to the Configuration and Build Guide.
2. You have already created a VSB project for your target CPU architecture and board support package (BSP).

Procedure

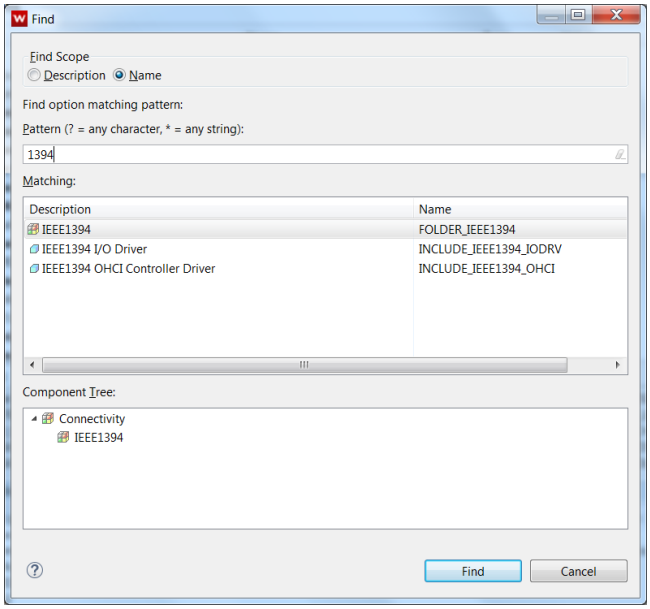
1. Add the 1394 layer to your VSB.
    1. Open the VSB Configuration in Workbench.
    2. Search for IEEE1394.



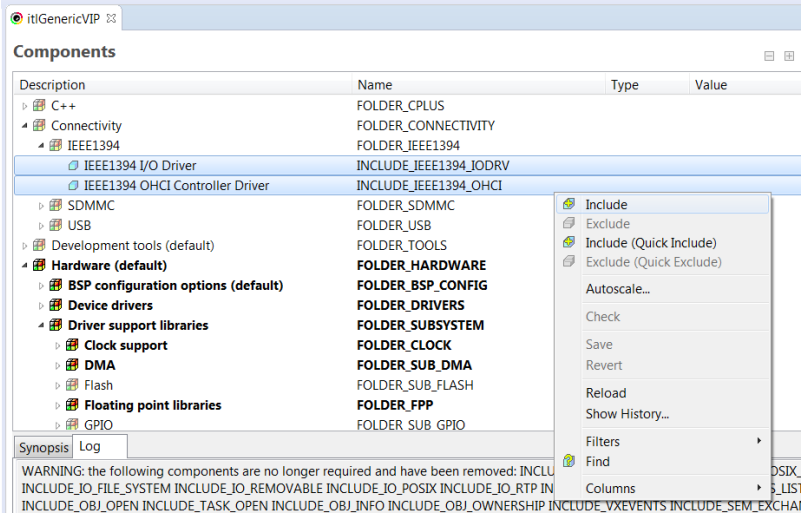
    3. Right-click on the layer and select Add with Dependencies.



2. Build the VSB.
3. Create a VIP based on the VSB.
4. Add the 1394 components to your VIP:
    1. Search for IEEE1394.



    2. Include the **IODRV** and **OHCI** components to the VIP.



5. Build the VIP.

Next Steps

For information about configuring VxWorks with debug information facilities for IEEE 1394 devices and buses, see Configuring VxWorks to Display Debug Messages on page 31.

3. DEVELOPING APPLICATIONS

1. Coding Applications

Coding applications to communicate with or control IEEE 1394 devices involves using standard I/O routines to open the device, send an ioclt( ) request, and close the device.

To communicate with an IEEE 1394 device:

1. Open the device using open( ).
2. Send the request using ioclt( ) with the appropriate command. See ioctl( ) Command Reference on page 9.
3. Close the device using close( ).

Application Code

```
Application Code
/* includes */

#include <WIND_BASE/target/src/hwif/ieee1394/ieee1394LibP.h>

/* functions */

STATUS fwGetVersion
    (
    int fd;
    STATUS rc = ERROR;
    FW_GET_VERSION getVers;
)
{
    fd = open("/fw0", O_RDWR, 0);
    if (fd <= 0)
        return ERROR;

    rc = ioctl(fd, FWIO_GET_VERSION, (void*) &getVers);
    if (rc != OK)
        goto exit;

    printf("Major: %d, minor: %d, build string: %s\n", getVers.major,
           getVers.minor, getVers.buildStr);

exit:
    close(fd);
return rc;
}


STATUS fwSetGapCount(void)
    {
    int fd;
    STATUS rc = ERROR;
    FW_SET_LOCAL_HOST_PROPERTIES setHolder;
    SET_LOCAL_HOST_PROP_GAP_COUNT gap;
```

```
    fd = open("/fw0", O_RDWR, 0);
    if (fd <= 0)
        return ERROR;

    setHolder.level = FW_SET_LOCAL_HOST_PROPERTIES_GAP_COUNT;
    setHolder.pInfo = (VOID*)&gap;
    gap.gapCountLowerBound = 7;

    rc = ioctl(fd, FWIO_SET_LOCAL_HOST_PROPERTIES, (void*) &setHolder);
    if (rc != OK)
        goto exit;

    printf("Set gap count done.\n");

exit:
    close(fd);
    return rc;
}


STATUS fwIsochSetChannelBandwidth(void)
    {
    int fd = -1;
    STATUS rc = ERROR;
    FW_ISOCH_ALLOCATE_BANDWIDTH abw;
    FW_ISOCH_QUERY_RESOURCES qr;
    FW_ISOCH_FREE_BANDWIDTH fbw;
    FW_ISOCH_SET_CHANNEL_BANDWIDTH scb;

    fd = open("/fw0", O_RDWR, 0);
    if (fd <= 0)
        {
        printf("open() error at line %d\n", __LINE__);
        return ERROR;
        }

    /* Allocate bandwidth */

    abw.bytesPerFrameRequested = 256;
    abw.speed = FW_SPEED_400;
    rc = ioctl(fd, FWIO_ISOCH_ALLOCATE_BANDWIDTH, (void*) &abw);
    if (rc != OK)
        {
        printf("ioctl() error at line %d\n", __LINE__);
        goto exit;
        }

    qr.speed = FW_SPEED_400;
    rc = ioctl(fd, FWIO_ISOCH_QUERY_RESOURCES, (void*) &qr);
    if (rc != OK)
        {
        printf("ioctl() error at line %d\n", __LINE__);
        goto exit;
        }

    printf("400: channels: 0x%016llx bandwidth: %d\n",
```

```
           qr.channelsAvailable, qr.bytesPerFrameAvailable);

    /* Change the bandwidth. */

    scb.pBandwidth = abw.pBandwidth;
    scb.maxBytesPerFrame = 128;
    rc = ioctl(fd, FWIO_ISOCH_SET_CHANNEL_BANDWIDTH, (void*) &scb);
    if (rc != OK)
        {
        printf("ioctl() error at line %d\n", __LINE__);
        }

    qr.speed = FW_SPEED_400;
    rc = ioctl(fd, FWIO_ISOCH_QUERY_RESOURCES, (void*) &qr);
    if (rc != OK)
        {
        printf("ioctl() error at line %d\n", __LINE__);
        goto exit;
        }

    printf("400: channels: 0x%016llx bandwidth: %d\n",
           qr.channelsAvailable, qr.bytesPerFrameAvailable);

    /* Free the bandwidth. */

    fbw.pBandwidth = abw.pBandwidth;
    rc = ioctl(fd, FWIO_ISOCH_FREE_BANDWIDTH, (void*) &fbw);
    if (rc != OK)
        {
        printf("ioctl() error at line %d\n", __LINE__);
        }

exit:
    close(fd);
    return rc;
    }
```

2. ioctl( ) Command Reference

VxWorks provides a set of ioctl( ) commands to manage 1394 devices.

These commands are listed in the table below.

You can also refer to *installDir* **/vxworks-7/pkgs_v2/connectivity/ieee1394-***x.x.x.x* **/stack/h/ieee1394Lib.h**.

Table 1. ioctl( ) Command Codes

ioctl( ) Command	Description
FWIO_ALLOCATE_ADDRESS_RANGE	Allocates addresses in the target's 1394 address space.
FWIO_ASYNC_LOCK	Performs an asynchronous lock operation on the device specified.

WNDRVR

ioctl( ) Command	Description
FWIO_ASYNC_READ	Performs an asynchronous read operation on the node specified.
FWIO_ASYNC_STREAM	Writes packets to an isochronous channel, during the asynchronous phase of the 1394 bus.
FWIO_ASYNC_WRITE	Performs an asynchronous write operation to the node specified.
FWIO_BUS_RESET	Initiates a 1394 bus reset.
FWIO_FREE_ADDRESS_RANGE	Releases an address range.
FWIO_GET_CONFIG_ROM	Returns the contents of the node's configuration ROM.
FWIO_GET_GENERATION_COUNT	Retrieves the current bus reset generation. The generation count must be passed in each asynchronous read, write, and lock request.
FWIO_GET_GUID	Retrieves the device's GUID.
FWIO_GET_LOCAL_HOST_INFO	Returns information about the local node.
FWIO_GET_MAX_SPEED_BETWEEN_DEVICES	Returns the maximum (simultaneous) transfer speed that can be used from the local node to a set of destination devices.
FWIO_GET_NODE_ADDRESS	Returns the node address for the device.
FWIO_GET_VERSION	Returns driver version information.
FWOHCI_IOCTL_BUS_SHOW	Show bus information.
FWOHCI_IOCTL_DEV_SHOW	Show information about the device that is attached to the bus.
FWOHCI_IOCTL_GET_CROM	Get the configuration ROM information.
FWOHCI_IOCTL_GET_MAXSPD	Get the maximum transfer speed that can be used between the local node and a set of destination devices.
FWIO_ISOCH_ALLOCATE_BANDWIDTH	Allocates isochronous bandwidth.
FWIO_ISOCH_ALLOCATE_CHANNEL	Allocates an isochronous channel.
FWIO_ISOCH_ALLOCATE_RX_RESOURCES	Allocates a resource handle for receive transactions from a given isochronous channel.
FWIO_ISOCH_ALLOCATE_TX_RESOURCES	Allocates a resource handle for transmit transactions over a given isochronous channel.

ioctl( ) Command	Description
FWIO_ISOCH_FREE_BANDWIDTH	Releases isochronous bandwidth.
FWIO_ISOCH_FREE_CHANNEL	Releases an isochronous channel.
FWIO_ISOCH_FREE_RESOURCES	Releases an isochronous RX or TX resource handle.
FWIO_ISOCH_FREE_RX_BUFFER	Frees an isochronous RX buffer.
FWIO_ISOCH_MODIFY_STREAM_PROPERTIES	Dynamically changed the properties of an allocated resource without releasing and reallocating the resource.
FWIO_ISOCH_QUERY_CURRENT_CYCLE_TIME	Returns the current isochronous cycle time.
FWIO_ISOCH_QUERY_RESOURCES	Returns the bandwidth and channels currently available on the 1394 bus.
FWIO_ISOCH_RECEIVE	Starts receiving from an isochronous channel.
FWIO_ISOCH_SET_CHANNEL_BANDWIDTH	Resets the bandwidth on an already allocated bandwidth handle.
FWIO_ISOCH_STOP	Stops receiving packets on an isochronous channel.
FWIO_ISOCH_TRANSMIT	Transmits isochronous packets.
FWIO_SET_LOCAL_HOST_PROPERTIES	Sets properties in the local node (target).

3. Data Structures Reference

The structures defined in **ieee1394Lib.h** can be used in your application.

FW_ADDRESS_OFFSET

```
typedef struct addressOffset
  {
    UINT16 offHigh;
    UINT32 offLow;
  } FW_ADDRESS_OFFSET;
```

offHigh

The high 16 bits of the IEEE 1394 node offset address.

offLow

The low 32 bits of the IEEE 1394 node offset address.

FW_ADDRESS_RANGE

```
typedef struct addressRange
  {
  UINT16 offHigh;
  UINT16 offLength;
  UINT32 offLow;
  } FW_ADDRESS_RANGE;
```

offHigh

The high 16 bits of the IEEE 1394 node offset address.

offLength

The length of the address range.

offHigh

The low 32 bits of the IEEE 1394 node offset address.

FW_ALLOCATE_ADDRESS_RANGE

```
typedef struct allocateAddressRange
  {
  FW_ADDRESS_RANGE range;
  UINT32        accessType;
  UINT32        notifyFlags;
  VOID*  pHandle;
  char    shmName[80];
  UINT32        shmSize;
  char    shmNotifyMsgQName[80];
  char    wrPktMsgQName[80];
  } FW_ALLOCATE_ADDRESS_RANGE;
```

addressRange

Specifies the address and length of the IEEE 1394 node offset address space.

accessType

Specifies the access type using one or more of the following flags:

Flag	Description
FW_ACCESS_FLAGS_TYPE_READ	Allocated addresses can be read.
FW_ACCESS_FLAGS_TYPE_WRITE	Allocated addresses can be written to.
FW_ACCESS_FLAGS_TYPE_LOCK	Allocated addresses can be the target of a lock operation.

notifyFlags

Specifies the notification flags. If the accessType is read or lock, one or more of the following flags must be set:

Flag	Description
FW_FLAGS_NOTIFY_NEVER	No notification.
FW_FLAGS_NOTIFY_AFTER_READ	Notify the application after carrying out an asynchronous read operation.
FW_FLAGS_NOTIFY_AFTER_LOCK	Notify the application after carrying out an asynchronous lock operation.

pHandle

Contains the handle used to free the address range.

shmName

Contains the name of the shared memory object.

shmSize

Contains the size of the shared memory object.

shmNotifyMsgQName

Contains the name of the notify public message queue.

wrPktMsgQName

Contains the name of the write packet public message queue.

FW_ASYNC_LOCK

```
typedef struct asyncLock
  {
  ADDRESS_OFFSET destAddress;
  UINT32         generation;
  UINT32         transType;
  UINT32         dataLength;
  UINT64         argValue;
  UINT64         dataValue;
  UINT64         oldDataValue;
  } FW_ASYNC_LOCK;
```

destAddress

The destination address.

generation

The generation number.

transType

Specifies which atomic transaction to execute on the IEEE 1394 node. The following functions are supported:

Flag	Description
FW_LOCK_TRANSACTION_MASK_SWAP	If the bit values in the target locations match the bit values in *argValue*, then the bits are set to the bit values in *dataValue*.
FW_LOCK_TRANSACTION_COMPARE_SWAP	If the value in the target locations match *argValue*, then update the locations with *dataValue*.
FW_LOCK_TRANSACTION_FETCH_ADD	The contents of the target locations are added to the dataValue and written back into the target locations.
FW_LOCK_TRANSACTION_LITTLE_ADD	Little endian version of **LOCK_TRANSACTION_FETCH_ ADD**.
FW_LOCK_TRANSACTION_BOUNDED_ADD	If the value of target locations do not equal the value of *argValue* then update the location of the sum of the old data plus *dataValue*.
FW_LOCK_TRANSACTION_WRAP_ADD	If the value of target locations do not equal the value of *argValue* then update the location of the sum of the old data value plus *dataValue* or else replace the old data value in the target location with *dataValue*.

dataLength

Length of data. May be 4, 8 or 16:

Data Length	argValue Size	dataValue Size	Description
4	0	4	Single argument (32-bit).
8	4	4	Two arguments (32-bit).
8	0	0	Single argument (64-bit).
16	8	0	Two arguments (64-bit).

dataValue

Argument value (32 or 64 bits).

argValue

Data value (32 or 64 bits).

oldValue

Old value (32 or 64 bits).

FW_ASYNC_READ

```
typedef struct asyncRead
```

```
  {
  ADDRESS_OFFSET        destAddress;
  UINT32        generation;
  VOID* pData;
  UINT32        length;
  } FW_ASYNC_READ;
```

destAddress

The destination address.

generation

The generation number.

pData

The pointer to the read buffer.

length

The number of bytes to read.

FW_ASYNC_STREAM

```
typedef struct asyncStream
  {
  UINT32        tag;
  UINT32        channel;
  UINT32        synch;
  UINT32        length;
  VOID*  pData;
  } FW_ASYNC_STREAM;
```

tag

Reserved. Set to 0.

channel

The isochronous channel number.

synch

Reserved. Set to 0.

length

The number of bytes to read.

pData

The pointer to the read buffer.

FW_ASYNC_WRITE

```
typedef struct asyncWrite
  {
  ADDRESS_OFFSET        destAddress;
  UINT32                generation;
  VOID*          pData;
```

```
  UINT32                length;
  } FW_ASYNC_WRITE;
```

destAddress

The destination address.

generation

The generation number.

pData

The number of bytes to write.

length

The number of bytes to write.

FW_BUS_RESET

```
typedef struct busReset
  {
  UINT32 flags;
  } FW_BUS_RESET;
```

flags

Set this flag to **FW_BUS_RESET_FLAGS_FORCE_ROOT** to make the local node the root node.

FW_CYCLE_TIME

```
typedef struct cycleTime
  {
  UINT32 cycleOffset : 12;
  UINT32 cycleCount : 13;
  UINT32 secondCount : 7;
  } FW_CYCLE_TIME;
```

cycleOffset

The count of 24.576MHz clocks modulo 3072.

cycleCount

The count of cycles (cycleOffset rollovers) modulo 8000.

secondCount

The count of seconds (*cycleCount* rollovers) modulo 128.

FW_FREE_ADDRESS_RANGE

```
typedef struct freeAddressRange
  {
  VOID* handle;
  } FW_FREE_ADDRESS_RANGE
```

handle

The handle to the address range returned in **FW_ALLOCATE_ADDRESS_RANGE**..

FW_GET_CONFIG_ROM

```
typedef struct getConfigRom
  {
  VOID*   pConfigRom;
  UINT32         length;
  } FW_GET_CONFIG_ROM
```

pConfigRom

Pointer to the configuration ROM buffer.

length

Length of the configuration ROM buffer in bytes, up to a maximum of 1 KB.

FW_GET_GENERATION_COUNT

```
typedef struct getGenerationCount
  {
  UINT32 generationCount;
  } FW_GET_GENERATION_COUNT;
```

generationCount

The current generation count.

FW_GET_LOCAL_HOST_INFORMATION

```
typedef struct getLocalHostInformation
  {
  UINT32        level;
  VOID* pInfo;
  } FW_GET_LOCAL_HOST_INFORMATION;
```

level

Specifies the level of information. Uses the following flags:

Flag	Description
FW_GET_HOST_UNIQUE_ID	Requests the controller's 64-bit unique identifier.
FW_GET_HOST_CAPABILITIES	Requests the controller's capability flags.
FW_GET_HOST_CONFIG_ROM	Requests the controller's configuration ROM.
FW_GET_HOST_PUBLIC_OBJECTS	Requests the driver's public objects.

pInfo

Points to the information block:

Flag	Data Structure
FW_GET_HOST_UNIQUE_ID	**FW_GET_LOCAL_HOST_UID**
FW_GET_HOST_CAPABILITIES	**FW_GET_LOCAL_HOST_CAPABILITIES**
FW_GET_HOST_CONFIG_ROM	**FW_GET_LOCAL_HOST_CROM**
FW_GET_HOST_PUBLIC_OBJECTS	**FW_GET_LOCAL_HOST_PUBLIC_OBJECTS**

FW_GET_LOCAL_HOST_UID and **FW_GET_GUID**

```
typedef struct getLocalHostGUID
  {
  UINT64        guid;
  } FW_GET_GUID, FW_GET_LOCAL_HOST_GUID;
```

guid

The node's GUID.

FW_GET_LOCAL_HOST_CAPABILITIES

```
typedef struct getLocalHostCapabilities
  {
  UINT32 maxAsyncReadRequest;
  UINT32 maxAsyncWriteRequest;
  } FW_GET_LOCAL_HOST_CAPABILITIES;
```

maxAsyncReadRequest

The maximum size asynchronous read request that the host controller supports.

maxAsyncWriteRequest

The maximum size asynchronous write request that the host controller supports.

GET_LOCAL_HOST_CROM

```
typedef struct getLocalHostCrom
  {
  VOID* pConfigRom;
  UINT32 configRomLength;
  } GET_LOCAL_HOST_CROM;
```

pConfigRom

Pointer to the beginning of the buffer to be filled with the local host's configuration ROM.

configRomLength

Specifies the length of the buffer pointed to by *pConfigRom*.

FW_GET_LOCAL_HOST_PUBLIC_OBJECTS

```
typedef struct getLocalHostPublicObjects
  {
```

```
  char busResetMsgQName[80];
  } FW_GET_LOCAL_HOST_PUBLIC_OBJECTS;
```

busResetMsgQName

Contains the name of the bus reset public message queue. A **FW_BUS_RESET_MSG** is sent to this queue after each bus reset.

FW_BUS_RESET_MSG

```
typedef struct busResetMsg
  {
  UINT32 generationCount;
  } FW_BUS_RESET_MSG;
```

generationCount

Generation count of the last bus reset.

FW_GET_MAX_SPEED_BETWEEN_DEVICES

```
typedef struct getMaxSpeedBetweenDevices
  {
  UINT32        numberOfNodes;
  NODE_ADDRESS destNodes[64];
  UINT32        maxSpeed;
  } FW_GET_MAX_SPEED_BETWEEN_DEVICES;
```

numberOfNodes

The number of nodes in *destNodes* .

destNodes

The set of nodes for which to find the maximum speed.

maxSpeed

The maximum speed between the nodes in *destNodes* .

FW_GET_NODE_ADDRESS

```
typedef struct getNodeAddress
  {
  NODE_ADDRESS nodeAddress;
  } FW_GET_NODE_ADDRESS;
```

nodeAddress

The node address (number).

FW_GET_VERSION

```
typedef struct getVersion
  {
  UINT8 major;
  UINT8 minor;
```

```
  char buildStr[80];
  } FW_GET_VERSION;
```

major

Major version number.

minor

Minor version number.

buildStr

Build string.

FW_ISOCH_ALLOCATE_BANDWIDTH

```
typedef struct isochAllocateBandwidth
  {
  UINT32        bytesPerFrameRequested;
  UINT32        speed;
  VOID*  pBandwidth;
  UINT32        bytesPerFrameAvailable;
  UINT32        speedSelected;
  } FW_ISOCH_ALLOCATE_BANDWIDTH;
```

bytesPerFrameRequested

Specifies the bandwidth requested in bytes per isochronous frame.

speed

Specifies the connection speed to use in allocating bandwidth. The possible values are:

Macro	Transfer Rate
FW_SPEED_100	100 Mb/s
FW_SPEED_200	200 Mb/s
FW_SPEED_400	400 Mb/s
FW_SPEED_800	800 Mb/s
FW_SPEED_1600	1600 Mb/s
FW_SPEED_3200	3200 Mb/s

pBandwidth

The handle to the allocated bandwidth.

bytesPerFrameAvailable

Bytes per frame available after the allocation attempt.

ThespeedSelected

The actual speed selected. The value is one of the **FW_SPEED**_*XXX* rates listed above.

ISOCH_ALLOCATE_CHANNEL

```
typedef struct isochAllocateChannel
  {
  UINT32        requestedChannel;
  UINT32        channel;
  UINT64        channelsAvailable;
  } ISOCH_ALLOCATE_CHANNEL;
```

requestedChannel

Specifies requested channel to allocate. Use **ISOCH_ANY_CHANNEL** for an arbitrary channel.

channel

The allocated channel if successful.

channelsAvailable

A bitmap specifying the available channels. The highest order bit (bit 63) specifies channel 0, the next bit (bit 62) specifies channel 1, and so on. This field is set even if the request fails.

FW_ISOCH_ALLOCATE_RX_RESOURCES

```
typedef struct isochAllocateRxResources
{
  UINT32        speed;
  UINT32        flags;
  UINT32        channel;
  UINT32        bufferSize;
  UINT32        numberOfBuffers;
  UINT32        watermark;
  VOID*  pResource;
  char    notifyMsgQName[80];
  char    shmName[80]; UINT32   shmSize;
  } FW_ISOCH_ALLOCATE_RX_RESOURCES;
```

speed

Specifies the connection speed. The possible values are:

Macro	Transfer Rate
FW_SPEED_100	100 Mb/s
FW_SPEED_200	200 Mb/s
FW_SPEED_400	400 Mb/s
FW_SPEED_800	800 Mb/s
FW_SPEED_1600	1600 Mb/s
FW_SPEED_3200	3200 Mb/s

flags

Specifies buffer flags. **RESOURCE_STRIP_ISOCH_HEADER** is supported and strips the header from the isochronous packet.

channel

The isochronous channel.

bufferSize

The size of each packet buffer.

numberOfBuffers

The number of packet buffers.

watermark

The number of contiguous packet buffers returned in each notify message.

pResource

A handle to the allocated resource.

notifyMsgQName

Contains the name of the receive public message queue. An **FW_ISOCH_RX_BUFFER_COMPLETE** is sent to this queue when

shmName

Contains the name of the public shared memory.

shmSize

Contains the size of the shared memory.

FW_ISOCH_RX_BUFFER_COMPLETE

```
typedef struct isochRxBufComplete
  {
  size_t               bufferOffset;
  size_t               bufferLength;
  VOID*         pContext;
  FW_CYCLE_TIME cycleTime;
  UINT32               droppedCount;
  } FW_ISOCH_RX_BUFFER_COMPLETE;
```

bufferOffset

The offset in shared memory of the first packet.

bufferLength

The length of the packets.

pContext

Contains a pointer to the RX buffer object which should be freed using **FW_ISOCH_FREE_RX_BUFFER**.

cycleTime

Contains the cycle of the last packet.

droppedCount

Contains the number of dropped packets.

WNDRVR

FW_ISOCH_FREE_RX_BUFFER

```
typedef struct fwIsochFreeRxBuffer
  {
  VOID* pContext;
  } FW_ISOCH_FREE_RX_BUFFER;
```

pContext

Pointer to RX buffer object to free.

FW_ISOCH_ALLOCATE_TX_RESOURCES

```
typedef struct isochAllocateTxResources
  {
  UINT32        speed;
  UINT32        channel;
  VOID*  pResource;
  UINT32        bufferSize;
  UINT32        numberOfBuffers;
  UINT32        watermark;
  char    notifyMsgQName[80];
  } FW_ISOCH_ALLOCATE_TX_RESOURCES;
```

speed

Specifies the connection speed. The possible values are:

Macro	Transfer Rate
FW_SPEED_100	100 Mb/s
FW_SPEED_200	200 Mb/s
FW_SPEED_400	400 Mb/s
FW_SPEED_800	800 Mb/s
FW_SPEED_1600	1600 Mb/s
FW_SPEED_3200	3200 Mb/s

channel

The isochronous channel.

bufferSize

The size of each packet buffer.

numberOfBuffers

The number of packet buffers.

watermark

The number of contiguous packet buffers to be transmitted by **FWIO_ISOCH_TRANSMIT**.

pResource

A handle to the allocated resource.

notifyMsgQName

Contains the name of the transmit public message queue. An **FW_ISOCH_TX_BUFFER_COMPLETE** is sent to this queue when watermark packets are have been transmitted.

FW_ISOCH_TX_BUFFER_COMPLETE

```
typedef struct isochTxBufComplete
  {
  VOID*         pBuffer;
  UINT32               bufferLength;
  FW_CYCLE_TIME cycleTime;
  UINT32               droppedCount;
  } FW_ISOCH_TX_BUFFER_COMPLETE;
```

pBuffer

The pointer to the packet buffer transmitted.

bufferLength

The length of the packet buffer.

cycleTime

Contains the cycle of the last packet.

droppedCount

Contains the number of dropped packets.

FW_ISOCH_FREE_BANDWIDTH

```
typedef struct isochFreeBandwidth
  {
  VOID* pBandwidth;
  } FW_ISOCH_FREE_BANDWIDTH;
```

pBandwidth

The handle to the bandwidth to free.

FW_ISOCH_FREE_CHANNEL

```
typedef struct isochFreeChannel
  {
  UINT32        channel;
  } FW_ISOCH_FREE_CHANNEL;
```

channel

The channel to free.

FW_ISOCH_FREE_RESOURCES

```
typedef struct isochFreeResource
  {
  VOID* pResource;
  } FW_ISOCH_FREE_RESOURCES;
```

pResource

A handle to the resource to be freed.

FW_ISOCH_RECEIVE

```
typedef struct isochReceive
  {
  VOID* pResource;
  } FW_ISOCH_RECEIVE;
```

pResource

Specifies the resource handle to use for the receive operation.

FW_ISOCH_MODIFY_STREAM_PROPERTIES

```
typedef struct isochModifyStreamProperties
  {
  VOID*  pResource;
  UINT64        channelMask;
  UINT32        speed;
  } FW_ISOCH_MODIFY_STREAM_PROPERTIES;
```

pResource

Specifies the resource handle.

channelMask

Specifies the allocated channel.

speed

Specifies the connection speed for the allocated channel. The possible values are:

Macro	Transfer Rate
FW_SPEED_100	100 Mb/s
FW_SPEED_200	200 Mb/s
FW_SPEED_400	400 Mb/s
FW_SPEED_800	800 Mb/s
FW_SPEED_1600	1600 Mb/s
FW_SPEED_3200	3200 Mb/s

FW_ISOCH_QUERY_CURRENT_CYCLE_TIME

```
typedef struct isochQueryCurrentCycleTime
  {
  CYCLE_TIME    cycleTime;
  } FW_ISOCH_QUERY_CURRENT_CYCLE_TIME;
```

cycleTime

The current cycle time.

FW_ISOCH_QUERY_RESOURCES

```
typedef struct isochQueryResource
  {
  UINT32        speed;
  UINT32        bytesPerFrameAvailable;
  UINT64        channelsAvailable;
  } FW_ISOCH_QUERY_RESOURCES;
```

speed

Specifies the connection speed for the allocated channel. The possible values are:

Macro	Transfer Rate
FW_SPEED_100	100 Mb/s
FW_SPEED_200	200 Mb/s
FW_SPEED_400	400 Mb/s
FW_SPEED_800	800 Mb/s
FW_SPEED_1600	1600 Mb/s
FW_SPEED_3200	3200 Mb/s

bytesPerFrameAvailable

The bytes per frame available.

channelsAvailable

A bitmap specifying the available channels. The highest order bit (bit 63) specifies channel 0, the next bit (bit 62) specifies channel 1, and so on.

FW_ISOCH_SET_CHANNEL_BANDWIDTH

```
typedef struct isochSetChannelBandwidth
  {
  VOID*  pBandwidth;
  UINT32        maxBytesPerFrame;
  } FW_ISOCH_SET_CHANNEL_BANDWIDTH;
```

pBandwidth

The handle to the allocated bandwidth.

maxBytesPerFrame

The new bandwidth.

FW_ISOCH_STOP

```
typedef struct isochStop
  {
  VOID* pResource;
  } FW_ISOCH_STOP;
```

pResource

Specifies the RX resource handle to stop.

FW_ISOCH_TRANSMIT

```
typedef struct isochTransmit
  {
  VOID* pResource;
  VOID* pBuffer;
  UINT32     length;
  } FW_ISOCH_TRANSMIT;
```

pResource

Specifies the resource handle to use for the transmit.

pBuffer

Specifies the packet buffer.

length

Specifies the length of the packet buffer.

NODE_ADDRESS

```
typedef struct addressRange
  {
  UINT16        nodeNumber: 6;
  UINT16        busNumber: 10;
  } FW_NODE_ADDRESS;
```

nodeNumber

The physical node number established by the PHY during self-identification.

busNumber

The number used to identify the specific 1394 bus that this node belongs to when multiple 1394-compatible buses are connected by way of a bridge.

FW_SET_LOCAL_HOST_PROPERTIES

```
typedef struct setLocalHostProperties
  {
  UINT32        level;
  VOID*         pInfo;
  } FW_SET_LOCAL_HOST_PROPERTIES;
```

level

Specifies what level of information should be set. The following flags are provided:

Flag	Description
FW_SET_LOCAL_HOST_PROPERTIES_GAP_COUNT	Sets a lower bound on the value the bus uses for its gap count.
FW_SET_LOCAL_HOST_PROPERTIES_MODIFY_CROM	Used to add or remove unit directories. This request cannot be used to modify instance directories or root keys in the configuration ROM.

pInfo

Points to the information block:

Flag	Data Structure
FW_SET_LOCAL_HOST_PROPERTIES_ GAP_COUNT	**FW_SET_LOCAL_HOST_PROP_GAP_COUNT**
FW_SET_LOCAL_HOST_PROPERTIES_ MODIFY_CROM	**SET_LOCAL_HOST_PROP_CROM**

FW_SET_LOCAL_HOST_PROP_GAP_COUNT

```
typedef struct setLocalHostPropGapCount
  {
  UINT32        gapCountLowerBound;
  } FW_SET_LOCAL_HOST_PROP_GAP_COUNT;
```

gapCountLowerBound

The lower bound of the gap count.

FW_SET_LOCAL_HOST_PROP_CROM

```
typedef struct setLocalHostPropCrom
  {
  UINT32        flags;
  VOID*        handle;
  UINT32        length;
  VOID*  pUnitDir;
  } FW_SET_LOCAL_HOST_PROP_CROM;
```

flags

Set to **FW_SLHP_FLAG_ADD_CROM_DATA** to create a new unit directory. Set to **FW_SLHP_FLAG_REMOVE_CROM_DATA** to remove an existing unit directory.

handle

Contains the handle that is returned after a unit directory has been created.

length

The length of the data buffer pointed to by *pUnitDir* .

pUnitDir

Pointer to the data buffer containing the unit directory information.

4. DEBUGGING AND TROUBLESHOOTING

1. Displaying Device Information with devs

You can display information about IEEE 1394 devices with the devs command.

About This Task

Each IEEE 1394 device (node) provides an entry in the VxWorks device table. The syntax for the device name is:

```
/fwc[:d]
```

where *fw* indicates that it is a 1394 device, *c* is the controller number (local node), and *d* is the device number.

Use the devs command from the shell to display information about devices.

Procedure

```
-> devs
drv name
0 /null
1 /ttyS0
...
11 /fw0
11 /fw0:0
11 /fw0:1
11 /fw1
11 /fw1:0
...
```

In this example, there are two controllers: the local nodes **/fw0** and **/fw1**. The two external nodes **/fw0:0** and **/fw0:1** are connected to local node **/fw0**, and the external node **/fw1:0** connected to local node **/fw1**.

2. ioctl() Information Commands for IEEE 1394

The core stack driver supports ioctl( ) commands that display information about the device and buses.

The ioctl( ) information commands are as follows:

- FWOHCI_IOCTL_BUS_SHOW
- FWOHCI_IOCTL_DEV_SHOW
- FWOHCI_IOCTL_GET_CROM
- FWOHCI_IOCTL_GET_MAXSPD

For general information about the VxWorks ioctl( ) commands used to manage 1394 devices, see ioctl( ) Command Reference on page 9.

3. Using IEEE 1394 Information Functions

The fwCardCountGet( ) and fwCardBusShow( ) functions can be used from the shell to display information about devices and buses.

About This Task

The use of shell information functions is illustrated in the following commands.

Procedure

1. Determine the number of cards attached to the system.

```
->fwCardCountGet
value = 3 = 0x3
```

   In this example, three cards were found.

2. Query information about one of the cards.

```
->fwCardBusShow
Node    GUID
----    ----
0       0x78563412000002ad:local
2       0x00133b0100010224
3       0x00133b010001022e:root,IRM,BM
------------------------
port count      gap count
3       8
3       8
3       8
value = 0 = 0x0
```

4. Configuring VxWorks to Display Debug Messages

You can configure the core stack driver to print debug messages for the device initialization phase and for bus activity.

About This Task

These steps configure VxWorks for debug messages and involve updating layer source files that are located in *installDir* **/vxworks-7/pkgs_v2/connectivity/ieee1394-***x.x.x.x* **/stack/src**:

Procedure

1. Edit **ieee1394DebugP.h** (which also lists the debug types), defining **FWIO_DEBUG**, as follows:

```
#undef FWIO_DEBUG
#define FWIO_DEBUG
```

2. Edit **ieee1394Ohci.c**, setting **fwIoDebug** as the debug type, as follows:

```
uint32_t fwIoDebug = DBG_INIT | DBG_BUS;
```

3. Rebuild the core stack driver and VxWorks.

   For more information, see Configuring VxWorks with 1394 Serial Bus Support on page 4.

5. Configuring VxWorks to Receive Asynchronous Stream Data

About This Task

You can configure VxWorks to receive asynchronous stream data on IEEE 1394 using the steps below.

Procedure

1. ioctl **FWIO_ISOCH_ALLOCATE_CHANNEL**
2. ioctl **FWIO_ISOCH_ALLOCATE_RX_RESOURCES**
3. shm_open shmName from **FW_ISOCH_ALLOCATE_RX_RESOURCES**
4. mmap shared mem from **FWIO_ISOCH_ALLOCATE_RX_RESOURCES** to process space
5. msgQOpen notifyMsgQName from **FWIO_ISOCH_ALLOCATE_RX_RESOURCES**
6. ioctl **FWIO_ISOCH_RECEIVE**
7. Create a task to wait on **notifyMsgQName**.
8. Gather data from the offset returned in **FW_ISOCH_RX_BUFFER_COMPLETE** into the mmap pointer.
9. ioctl **FWIO_ISOCH_FREE_RX_BUFFER**, continue

6. Error Code Reference

VxWorks provides error codes specific to the 1394 serial bus, to which the global errno may be set.

Table 1. Error Codes

Error Code	Description
S_fwLib_BUS_RESET	A bus reset has occurred.
S_fwLib_DEVICE_ERROR	A device error has occurred.
S_fwLib_INVALID_ARG	Invalid argument.